Machine Learning Course - MiniProject II

Matin Moradi

Åbo Akademi University
Matin.Moradi@abo.fi

Introduction

Imagine a large-scale test such as TOEFL or IELTS and thousands of essays that should be scored. If humans want to assess them all manually, it's a very difficult task. Now what if an automatic system can do it in a few seconds/minutes? That's what we're aiming in this project which is known as Automated Essay Scoring (AES) task. It's a supervised NLP problem in which the model tries to predict an ordinal number for each essay as its score. This is an important task becasue it provides a scalable, consistent, and cost-effective solution for grading large volumes of writing while eliminating human rater bias. Furthermore, it offers students immediate, actionable feedback.

In this problem, there are around 17k essays associated with a score between 1-6. The goal is to try two different approaches to predict the scores.

Data Analysis and Processing

First I loaded the data which includes 17307 data rows and 3 columns (essay_id, full_text, and score) in one csv file. There were no NULL value or duplicate rows in the data. First I looked at the distribution of the target value in Figure1a which is close to a normal distribution that picks at 3. We can also see the distribution of word count in essays in Figure1b where the mean is 358 words and most of them have 200-600 words as it was indicated in the project description. We can then ask: "is there any relationship between the average word length in an essay with the score it gets?" To answer this, we can look at Figure1c which shows a positive correlation between these two. So, the more complex the words in an essay are, the higher the probability that it will receive a better score. But how about the total length of the essay. Generally, "do short concise essays get a better number or the lengthy ones?" Figure1d indicates that longer essays can get higher scores. Overall, we can see that a higher word count and the use of more complex words increase the chances of receiving a better score.
Following the initial overview, I performed a stratified 80/20 split to ensure that the distribution of the target variable remained consistent across both the training and testing sets.

Feature Engineering

Classical (feature-based) features

For the classical baseline, I combined three types of features to capture lexical, subword, and structural information from the essays:

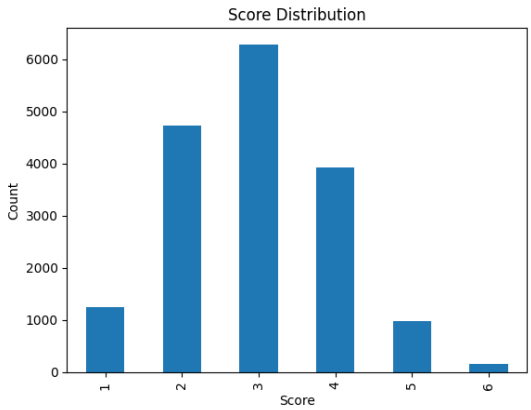
- **Word-level TF-IDF:** ngrams (1–2), unigrams and bigrams actually, with a maximum of 50,000 features and English stop word removal. This captures important words and short phrases related to writing quality.
- **Character-level TF-IDF:** Character n-grams (3–5) with up to 30,000 features. This can represent spelling, morphology, and the style of the essays.
- **Length-based features:** According to the results from the EDA part, length and complexity related features are important. Therefore, I considered these features for each essay: number of words, number of unique words, and the average length of a word.

All feature groups were combined into a single representation and used as input to the Ridge regression model.

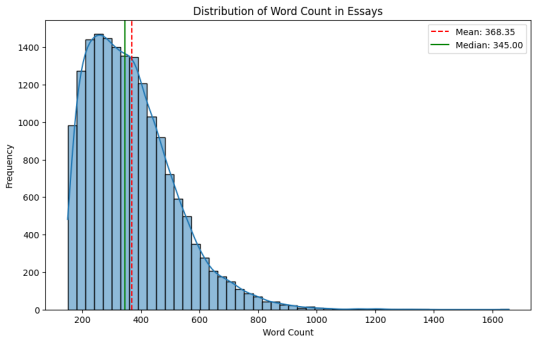
Neural features (BiLSTM input)

For the neural model, feature engineering was performed differently compared to the classical approach. Instead of manually designing TF-IDF and shallow features, we need to convert the raw text into sequences of token indices and allow the model to learn useful representations automatically.

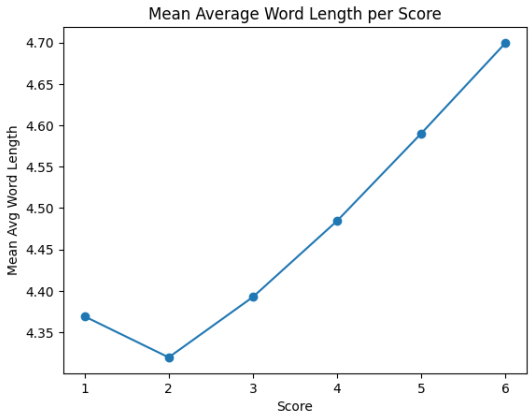
- **Vocabulary Construction:** A vocabulary was built from the training data using the 20,000 most frequent words. Two special tokens were added: `<PAD>` for padding and `<UNK>` for unknown words. Each word was mapped to a unique integer index.
- **Token Encoding and Truncation:** Each essay was tokenized and converted into a sequence of word indices. The maximum sequence length was limited to 500 tokens. Longer essays were truncated, and shorter ones were padded.
- **Learned Word Embeddings:** Each word index was mapped to a dense embedding vector of size 100. These embeddings were learned during training which allows the model to capture semantic relationships between words.



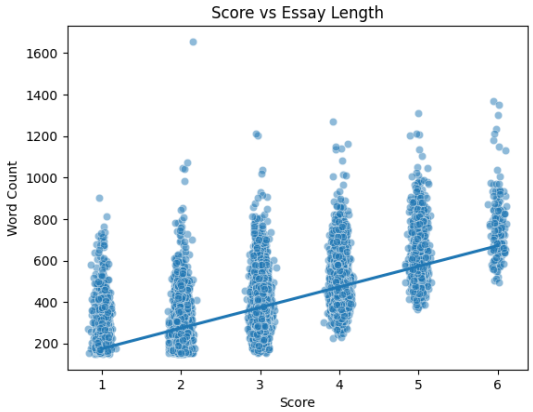
(a) Score distribution



(b) Word count distribution



(c) Avg. word length vs score



(d) Essay length vs score

Figure 1: Exploratory data analysis of essay scores and length features.

Modeling

Evaluation metric

Primary metric: **Quadratic Weighted Kappa (QWK)**. QWK is appropriate for ordinal labels because it penalizes larger disagreements more.
Secondary metrics reported: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Spearman correlation.

Classical baseline: Ridge

TF-IDF (word n-grams) + TF-IDF (char n-grams) + shallow features were used. Then StandardScaler was also applied. Ridge model has an alpha parameter which I tuned it to 1 using 5-fold cross validation.

Neural: BiLSTM

In addition to the classical baseline, I implemented a neural model based on a Bidirectional Long Short-Term Memory (BiLSTM) network with an attention mechanism.

Model Architecture For setting the input maximum sequence length, I referred to Figure 1b, which shows that the average essay length is 368 words. I first considered `MAX_LEN = 400` in the model, which resulted in `mean_qwk = 0.686` in cross-validation. Then, I increased it to `MAX_LEN = 500`, and `mean_qwk` improved to 0.704.

I also changed the `HIDDEN_DIM` parameter from 128 to 256, which led to a decline in QWK from 0.686 to 0.655. Similarly, I increased `EMBEDDING_DIM` from 100 to 200, which reduced QWK from 0.686 to 0.678.

Based on these experiments, I selected `MAX_LEN = 500` for the maximum sequence length, while keeping `HIDDEN_DIM = 128` and `EMBEDDING_DIM = 100`.

After setting those parameters and obtaining QWK=0.704 for the model, the main improvement was adding the attention pooling and early stopping. It increased the mean CV QWK to 0.761 which means around 6% improvement. The following shows the final architecture of the model:

- Word embeddings (dim=100) were learned during training.
- A bidirectional LSTM (hidden dim = 128) processed the sequence from both directions.
- An attention layer computed a weighted sum of hidden states. It allows the model to focus on important words.
- To reduce the overfitting, a dropout layer (0.3) was applied.
- The final layer produced a single continuous score.

Training and Evaluation

- The model was trained using the Adam optimizer with learning rate 0.001.
- Mean Squared Error (MSE) was used as the loss function.
- Early stopping based on QWK
- Model selection using 5-fold stratified cross-validation.
- Final evaluation on the test set included QWK, MAE, RMSE, and Spearman correlation.

Results

Classical baseline: Ridge

I used 5-fold cross validation, applied feature ablation, and analyzed the results. Then I trained the model once on the entire training dataset and finally on the test dataset. Mean QWK for the CV phase was 0.6933. You can see the effect of removing each feature on the final result in Figure2 and Table1 which show that word_tfidf and then char_tfidf have the most contribution in the result.

Table 1: Ablation Study Results for Ridge Baseline Model

Model Configuration	Mean CV QWK
Full Ridge Model (Baseline)	**0.6933**
– without $word_tfidf$	0.5343
– without $char_tfidf$	0.6312
– without $length_features$	0.6816

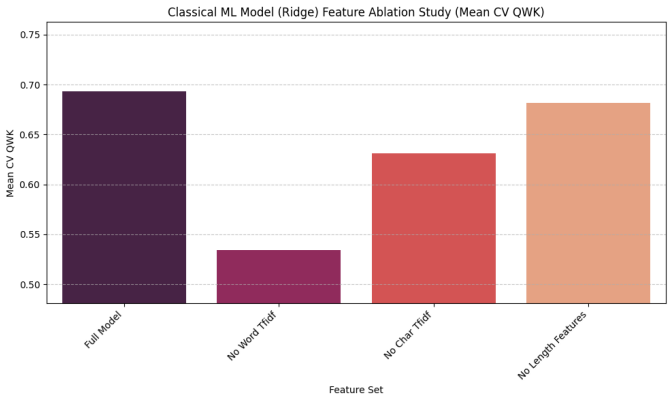


Figure 2: Classical Feature Ablation

We can also analyze the effect of n_grams and each length feature in more detail by visualizing their coefficients learned by the Ridge model. As demonstrated in the feature importance Figure3 word_count and unique_word_count have the highest coefficients and other features have much lower numbers.

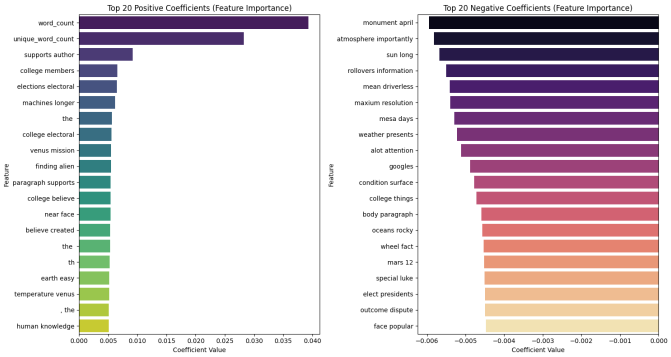


Figure 3: Ridge Feature Importance

Finally the results on the test data are shown in Table2

Metric	Value
QWK	0.6964
MAE	0.5764
RMSE	0.7194
Spearman Correlation	0.7203

Table 2: Test set performance of the Ridge model.

Neural: BiLSTM

Similar to the ridge model, I applied 5-fold CV, then the main training phase, and lastly the testing phase. An overview of the results can be seen in Table3 which shows strictly improved scores compared to the ridge baseline.

Table 3: BiLSTM Model Performance Metrics

Phase	Mean QWK
5-Fold Cross-Validation	0.7568
Main Training	0.7614
Final Test	**0.7340**

The main training phase continued to 8 epochs although it was halted by the early stopping technique since the best performance was in the fifth epoch. All the primary and secondary metrics for the test performance are shown in Table4

Table 4: Final BiLSTM Test Set Evaluation Metrics

Metric	Value
Quadratic Weighted Kappa (QWK)	**0.7340**
Mean Absolute Error (MAE)	0.5358
Root Mean Square Error (RMSE)	0.6854
Spearman Correlation (ρ)	0.7817

data-size learning curve For four times, I did 3-fold CV on the training data but each time on a different data size, 20%, 40%, 60%, and 80% of the data to see how the training size can affect the scores. As shown in Figure4 using 80% of the data was a good choice as the performance decreases using less amount of data. It shows that the model benefits significantly from more data without being overfitted.

Token Attribution To analyze model interpretability and understand how the model works I looked at which parts of the text it focuses on when making predictions. I did this by selecting five random essays and highlighting the words in the text that the model pays more attention to. One of them can be seen in Figure5. The model tends to focus on the meaningful words and phrases that make an argument (such as 'dangers' or 'despite') and it ignores the small words that do not add much meaning. This tells us that the model more focuses on semantically important parts of the text when it makes a prediction.

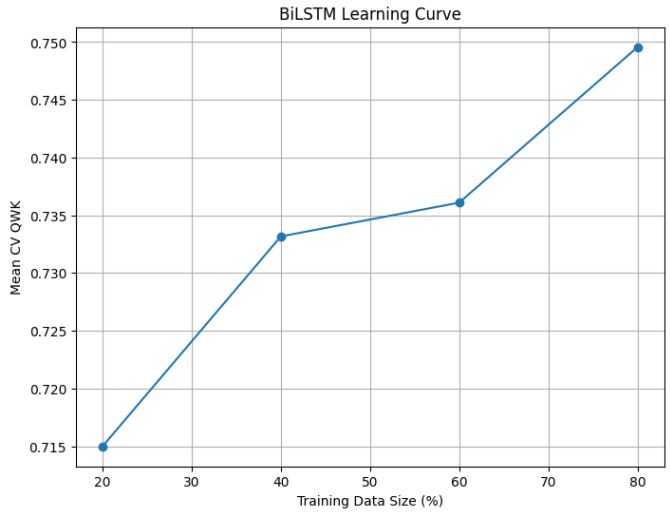


Figure 4: What could have happened if we used less data?

I also wanted to see if the model focuses on parts of the essay more than others. So I looked at how the model's attention changes throughout the text. Figure6 shows relatively stable attention across most of the essay, with a clear increase near the final 5% of the text. This makes sense because it is the conclusion part of the essay where the writers sums up their points.

Figure 5: Token Attention: Words with higher attention were highlighted with stronger color intensity

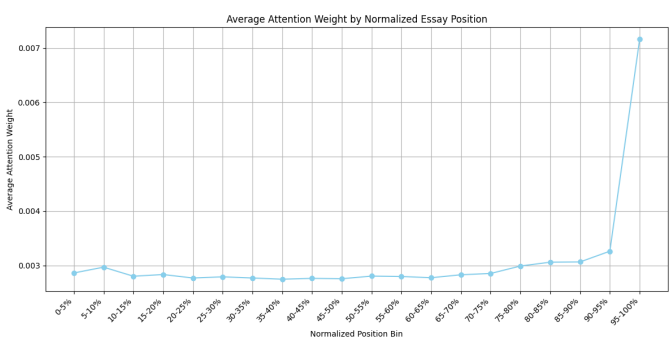


Figure 6: Attention vs position in the text

Statistical Testing To compare the two models I did a bootstrap analysis on the cross-validation QWK differences. I looked at the differences in QWK scores between the BiLSTM model and the Classical ML model. The BiLSTM model was 0.0635 better on average. As shown in Figure7, The range of improvements, which is called the 95% confidence interval was between 0.0505 and 0.0742. Since this range is above zero we can say that BiLSTM model is significantly better than the ML model. This means that the BiLSTM model works better than the ML model in a consistent way and its not just luck. The improvement of BiLSTM, over Classical ML is real.

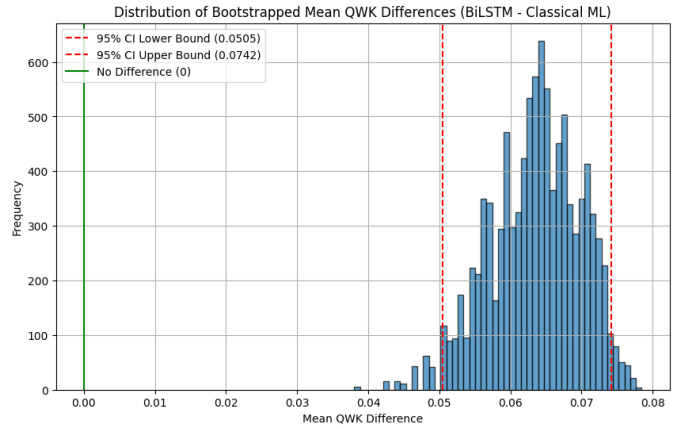


Figure 7: The 95% confidence interval is entirely above zero

Conclusion

In this project, I worked on an Automated Essay Scoring system using two models: Ridge regression and a BiLSTM neural network. I first studied 17,307 essays and prepared features like TF-IDF and essay length for the Ridge model. For the BiLSTM, I converted essays into word sequences with embeddings and added an attention layer. I tested both models using QWK, MAE, RMSE, and Spearman correlation, and also looked at feature importance and attention patterns. The BiLSTM performed better than Ridge which shows that it can understand essay meaning and structure more effectively.

One of the main bottlenecks was finetuning the parameters to get a desired performance without overfitting. For example, initially I was getting very low results from the Ridge model (under 50%) and then I realized the reason is the tf_idf feature size which were 10k and 5k for word-level and char-level tf_idf. I increased them to 50k and 30k hence the QWK score improved by 20% demonstrating that the feature size was the main bottleneck.

A possible improvement can be using pretrained word embeddings like GloVe instead of starting with random embeddings for the initialization. Pretrained embeddings are already trained on large text collections and capture the meaning and relationships between words